Date:20/08/2026

2. Multiple linear Regression model

Procedure:

Build a Multiple Linear Regression model and check its parametric assumptions such as linearity, independence, normality, multicollinearity, and homoscedasticity.

1. Data Preparation:

- The dataset is loaded and split into independent features (X) and dependent variable (y).
- The dataset is further divided into a training and test set.

2. Model Building:

- A multiple linear regression model is trained on the training set using the LinearRegression() model from sklearn.
- After training, the model's coefficients and intercept are printed, followed by prediction on the test data and model evaluation using metrics like Mean Squared Error and R-squared.

3. Parametric Assumptions:

- Linearity: Checked by plotting Residuals vs Fitted Values.
- Normality of Residuals: Verified using a Q-Q plot and Shapiro-Wilk test.
- Homoscedasticity: Tested by checking if residuals have constant variance through Residuals vs Fitted Values plot.
- Multicollinearity: Assessed by calculating Variance Inflation Factor (VIF) for each predictor. A VIF above 10 indicates multicollinearity.
- Independence of Residuals: Evaluated using the Durbin-Watson test to check for autocorrelation. A value between 1.5 and 2.5 indicates no autocorrelation.

4. Outputs and Interpretations

- Model Summary: Provides detailed statistics, including coefficients, p-values, R-squared, adjusted R-squared, and more.
- Residual Diagnostics: Helps verify assumptions like normality, homoscedasticity, and linearity.
- VIF: Identifies if there is multicollinearity between predictors.
- Durbin-Watson: Ensures residuals are independent (i.e., no autocorrelation).

Solution:

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
from statsmodels.stats.outliers_influence import variance_inflation_factor
import scipy.stats as stats
```

```
df = pd.read_csv(r"C:\Users\bhumi\Downloads\winequality-white - winequality-white.csv")
```

```
print(df.columns.tolist())
```

```
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH'
hates', 'alcohol', 'quality']
```

```
X = df.drop('quality', axis=1)
y = df['quality']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

LinearRegression

LinearRegression()

```
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

y_pred = model.predict(X_test)

mse = metrics.mean_squared_error(y_test, y_pred)
r2 = metrics.r2_score(y_test, y_pred)

print("\nModel Evaluation")
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Intercept: 124.39391497104634
Coefficients: [ 4.59072370e-02 -1.91488432e+00 -6.13034699e-02  7.12395507e-02
 -2.64751329e-02  5.11945372e-03 -2.42153962e-04 -1.24264125e+02
  6.00699854e-01  6.49072780e-01  2.29008684e-01]

Model Evaluation
Mean Squared Error: 0.5690247717275294
R-squared: 0.2652750042119709
```

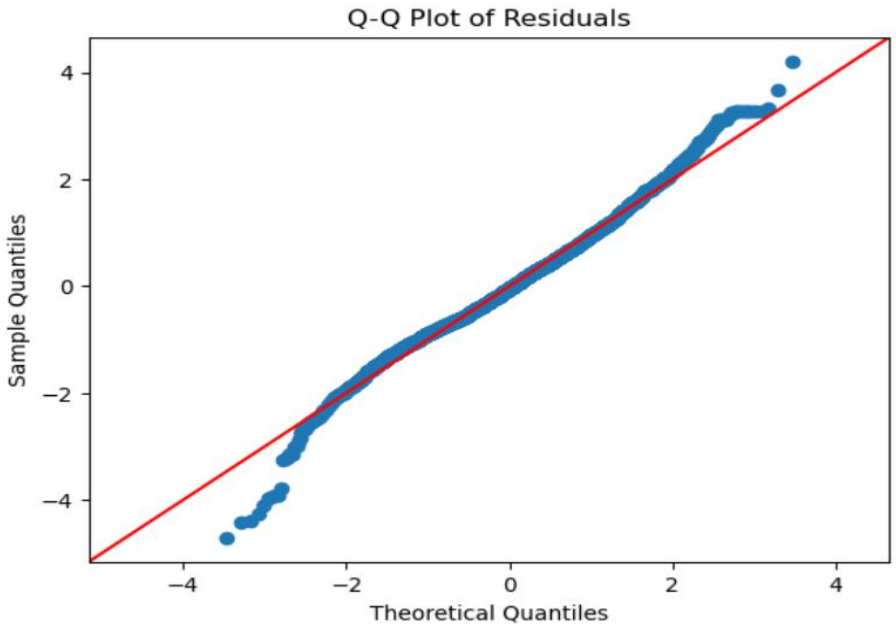
```
fitted_values = ols_model.fittedvalues
residuals = ols_model.resid

plt.figure(figsize=(8, 6))

sm.qqplot(residuals, line="45", fit=True)

plt.title("Q-Q Plot of Residuals")
plt.show()
```

```
<Figure size 800x600 with 0 Axes>
```



```
X_train_const = sm.add_constant(X_train)

# Fit OLS model
ols_model = sm.OLS(y_train, X_train_const).fit()

print("\nOLS MODEL SUMMARY")
print(ols_model.summary())
```

```
OLS MODEL SUMMARY
                            OLS Regression Results
==============================================================================
Dep. Variable:                quality   R-squared:                       0.284
Model:                            OLS   Adj. R-squared:                  0.282
Method:                 Least Squares   F-statistic:                     141.1
Date:                Thu, 20 Aug 2026   Prob (F-statistic):          8.15e-274
Time:                        12:18:57   Log-Likelihood:                -4433.4
No. Observations:                3918   AIC:                             8891.
Df Residuals:                    3906   BIC:                             8966.
Df Model:                          11
Covariance Type:            nonrobust
```

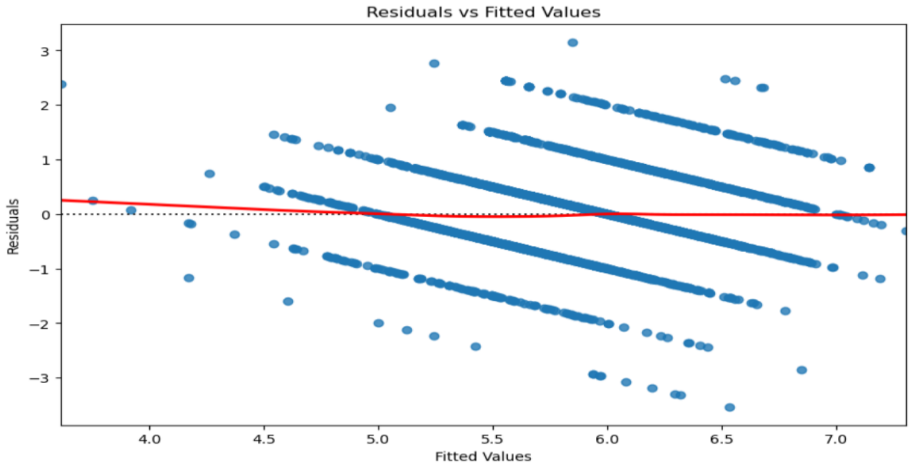
	coef	std err	t	P>|t|	[0.025	0.975]
const	124.3939	20.211	6.155	0.000	84.770	164.018
fixed acidity	0.0459	0.023	1.981	0.048	0.000	0.091
volatile acidity	-1.9149	0.126	-15.186	0.000	-2.162	-1.668
citric acid	-0.0613	0.108	-0.566	0.572	-0.274	0.151
residual sugar	0.0712	0.008	8.687	0.000	0.055	0.087
chlorides	-0.0265	0.610	-0.043	0.965	-1.223	1.170
free sulfur dioxide	0.0051	0.001	5.327	0.000	0.003	0.007
total sulfur dioxide	-0.0002	0.000	-0.568	0.570	-0.001	0.001
density	-124.2641	20.511	-6.059	0.000	-164.477	-84.052
pH	0.6007	0.118	5.106	0.000	0.370	0.831
sulphates	0.6491	0.113	5.766	0.000	0.428	0.870
alcohol	0.2290	0.026	8.749	0.000	0.178	0.280

```
==============================================================================
Omnibus:                       88.824   Durbin-Watson:                   2.020
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              181.749
Skew:                           0.112   Prob(JB):                    3.42e-40
Kurtosis:                       4.031   Cond. No.                    3.59e+05
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.59e+05. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
print("\nAssumption 1: Linearity")
fitted_values = ols_model.fittedvalues
residuals = ols_model.resid
plt.figure(figsize=(10, 6))
sns.residplot(x=fitted_values, y=residuals, lowess=True, line_kws={"color": "red"})
plt.title("Residuals vs Fitted Values")
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.show()
```

```
Assumption 1: Linearity
```

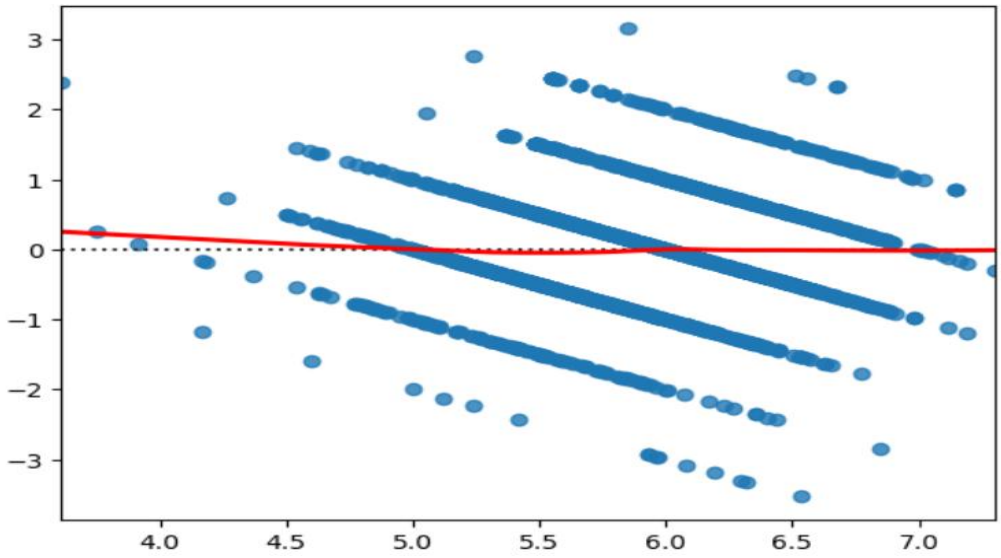


```
sns.residplot(
    x=fitted_values,
    y=residuals,
    lowess=True,
    line_kws={"color": "red"}
)
shapiro_test = stats.shapiro(residuals)

print("\nAssumption 2: Normality of Residuals")
print("Shapiro-Wilk Statistic:", shapiro_test.statistic)
print("Shapiro-Wilk p-value:", shapiro_test.pvalue)

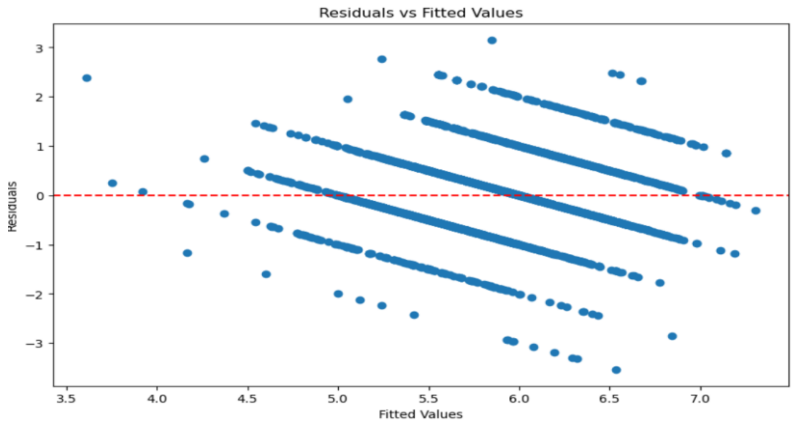
if shapiro_test.pvalue > 0.05:
    print("Conclusion: Residuals are approximately normally distributed.")
else:
    print("Conclusion: Residuals are not normally distributed.")
```

```
Assumption 2: Normality of Residuals
Shapiro-Wilk Statistic: 0.9898491183841173
Shapiro-Wilk p-value: 3.768474717323542e-16
Conclusion: Residuals are not normally distributed.
```




```
print("\nAssumption 3: Homoscedasticity")
plt.figure(figsize=(10, 6))
plt.scatter(fitted_values, residuals)
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Residuals vs Fitted Values')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.show()
```

```
Assumption 3: Homoscedasticity
```



```
print("\nAssumption 4: Multicollinearity")
vif_data = pd.DataFrame()
vif_data["Feature"] = X_train.columns
vif_data["VIF"] = [variance_inflation_factor(X_train_const.values, i+1)
                   for i in range(len(X_train.columns))]
print("VIF for each predictor:\n", vif_data)
print("\nAssumption 5: Independence of Residuals")
durbin_watson_stat = sm.stats.durbin_watson(residuals)

print(f"Durbin-Watson Statistic: {durbin_watson_stat}")

if 1.5 <= durbin_watson_stat <= 2.5:
    print("Residuals are independent (no autocorrelation).")
else:
    print("Residuals may be autocorrelated.")
```

```
Assumption 4: Multicollinearity
VIF for each predictor:
                  Feature        VIF
0           fixed acidity   2.657828
1        volatile acidity   1.138935
2             citric acid   1.168885
3          residual sugar  12.322910
4               chlorides   1.227690
5     free sulfur dioxide   1.782199
6    total sulfur dioxide   2.230122
7                 density  26.650522
8                      pH   2.165799
9               sulphates   1.134336
10                alcohol   7.167193

Assumption 5: Independence of Residuals
Durbin-Watson Statistic: 2.0200753149004638
Residuals are independent (no autocorrelation).
```